Informe Laboratorio 4

Sección 1

Felipe Nicolas Bustos López
e-mail: felipe.bustos@mail.udp.cl

Noviembre de 2026

Índice

1. Descripción **2**

2. Actividades **2**
2.1. Configuración y análisis de SSH . . . 2
2.1.1. Implementación y configuración del servidor SSH . . . 2
2.1.2. Análisis de paquetes SSH en Wireshark . . . 2
2.2. Análisis de captura de tráfico antigua . . . 3
2.2.1. Análisis de los mecanismos criptográficos . . . 3
2.2.2. Recuperación del PIN y derivación de la clave AES . . . 3
2.2.3. Descifrado de las transacciones y análisis final . . . 3

3. Desarrollo de actividades **4**
3.1. Actividad 1 . . . 4
3.1.1. Dockerfile . . . 4
3.1.2. Configuración del servidor . . . 5
3.1.3. Explicación de los algoritmos utilizados . . . 6
3.1.4. Conexión cliente-servidor . . . 6
3.1.5. Análisis de tráfico . . . 7
3.2. Actividad 2 . . . 11
3.2.1. Identificación del paquete RSA_PIN . . . 11
3.2.2. Identificación de los paquetes AESGCM . . . 13
3.2.3. Recuperación del PIN y derivación de la clave AES . . . 14
3.2.4. Descifrar los mensajes . . . 16
3.2.5. Análisis respecto de criptografía post-cuántica . . . 20

4. Bibliografia **21**

1. Descripción

Usted trabaja en una empresa especializada en la transferencia segura de información, archivos y contenido digital. Antes de incorporarse al desarrollo y operación de los sistemas reales de la organización, se le solicita realizar una demostración técnica que permita evaluar sus conocimientos en redes, protocolos de comunicación y criptografía.

Para ello, deberá implementar y configurar un servidor SSH, definiendo los mecanismos criptográficos utilizados para proteger la comunicación. Posteriormente, deberá establecer una conexión con el servidor y capturar el tráfico de red utilizando Wireshark.

A partir del análisis de los paquetes capturados, deberá identificar qué información puede ser observada por un tercero que tuviera acceso al tráfico de red y determinar qué elementos de la comunicación están protegidos mediante cifrado y cuáles pueden ser visibles, con el objetivo de evaluar el nivel de seguridad proporcionado por el protocolo SSH.

Finalmente, se le proporcionará una captura de tráfico de red obtenida mediante Wireshark. En ella se encuentran diversos mensajes correspondientes a transacciones que han sido cifrados mediante RSA.

El objetivo de esta actividad es analizar la captura de tráfico, identificar los paquetes que contienen la información cifrada y desarrollar un programa en Python que permita recuperar el contenido original.

2. Actividades

2.1. Configuración y análisis de SSH

2.1.1. Implementación y configuración del servidor SSH

Generar un contenedor Docker que permita ejecutar un servidor SSH accesible desde el equipo cliente (su computador). Posteriormente, configure el servidor mediante un archivo `sshd_config`, en el cual deberá especificar los algoritmos criptográficos utilizados durante la comunicación SSH: `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Por último, documente la función que cumple cada algoritmo dentro del proceso de establecimiento y comunicación de una conexión SSH.

2.1.2. Análisis de paquetes SSH en Wireshark

Una vez completada la configuración del servidor SSH, deberá analizar el tráfico de red generado durante el establecimiento de la conexión utilizando Wireshark.

Para ello, antes de realizar la conexión SSH, inicie la captura de tráfico en la interfaz de red correspondiente. Posteriormente, establezca una conexión con el servidor utilizando su máquina como cliente y analice los paquetes intercambiados durante el proceso de conexión.

A partir del análisis realizado, responda la siguiente pregunta:

¿Qué información es posible identificar a partir de los paquetes capturados? Mencione y explique al menos tres elementos que puedan observarse en el tráfico de la conexión SSH.

2.2. Análisis de captura de tráfico antigua

2.2.1. Análisis de los mecanismos criptográficos

Se le solicita analizar una captura de tráfico de red antigua, en formato .pcap, que contiene transacciones financieras cifradas mediante una combinación de criptografía asimétrica y simétrica. Para realizar la actividad, se le proporcionarán la captura y las claves pública y privada RSA correspondientes. Deberá identificar los paquetes relevantes y analizar el mecanismo de cifrado utilizado, considerando que RSA se emplea para cifrar un PIN de cuatro dígitos mediante OAEP con SHA-256, mientras que AES-GCM se utiliza para proteger el contenido de las transacciones financieras.

2.2.2. Recuperación del PIN y derivación de la clave AES

El primer paquete relevante de la captura contiene la etiqueta RSA_PIN, seguida de una salt de 16 bytes y del PIN cifrado mediante RSA. Deberá identificar y extraer estos campos, utilizar la clave privada RSA proporcionada para recuperar el PIN y, posteriormente, derivar la clave AES de 256 bits mediante Scrypt. Para ello, deberá utilizar el PIN recuperado, la salt extraída de la captura y los siguientes parámetros: (n=2**14), (r=8) y (p=1). Explique la función de la salt y por qué es necesario utilizar los mismos parámetros para obtener la clave AES original.

2.2.3. Descifrado de las transacciones y análisis final

Una vez obtenida la clave AES, deberá identificar todos los paquetes que contienen transacciones cifradas mediante AES-GCM. Para cada paquete, deberá extraer el nonce de 12 bytes y utilizarlo junto con la clave derivada y el texto cifrado para recuperar los mensajes originales, verificando su autenticidad. Implemente el procedimiento completo en un script de Python y presente las transacciones recuperadas en su formato original, incluyendo el identificador, el remitente, el destinatario y el monto. Documente el procedimiento realizado e incluya evidencias de los resultados obtenidos.



Figura 1: Ejemplo del resultado

Finalmente, responda la siguiente pregunta, justificando su respuesta: ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC (Criptografía post-cuántica)?

3. Desarrollo de actividades

3.1. Actividad 1

3.1.1. Dockerfile

Para implementar el servidor SSH solicitado se utilizó Docker. Esto permitió trabajar sobre un entorno aislado y reproducible, evitando modificar directamente la configuración SSH del computador utilizado como cliente.

Se utilizó Ubuntu 24.04 como imagen base y se instaló OpenSSH Server. Además, se creó un usuario destinado a las pruebas del laboratorio y se generó una clave de host Ed25519.

El archivo `Dockerfile` utilizado fue el siguiente:

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
   apt-get install -y openssh-server && \
   rm -rf /var/lib/apt/lists/*

RUN mkdir -p /run/sshd

RUN useradd -m -s /bin/bash felipe && \
   echo "felipe:lab04ssh"| chpasswd

RUN rm -f /etc/ssh/ssh_host_* && \
   ssh-keygen -t ed25519 \
   -f /etc/ssh/ssh_host_ed25519_key \
   -N

COPY sshd_config /etc/ssh/sshd_config

EXPOSE 22

CMD [/usr/sbin/sshd", D", e"]
```

La imagen fue construida desde la carpeta principal del laboratorio mediante:

```
docker build -t lab04-ssh .
```

Una vez creada la imagen se inició el contenedor, asociando el puerto 2222 del computador con el puerto 22 utilizado internamente por SSH:

```
docker run -d \
   --name lab04-ssh-server \
   -p 2222:22 \
   lab04-ssh
```

De esta forma, el servidor quedó disponible desde el equipo cliente mediante `localhost:2222`.

3.1.2. Configuración del servidor

La configuración del servicio SSH se realizó mediante un archivo `sshd_config` personalizado. En él se limitaron explícitamente los algoritmos disponibles con el objetivo de utilizar los mecanismos solicitados en la actividad.

La configuración utilizada fue:

```
Port 22
ListenAddress 0.0.0.0
PermitRootLogin no
PasswordAuthentication yes
PermitEmptyPasswords no
UsePAM no
AllowUsers felipe
HostKey /etc/ssh/ssh_host_ed25519_key
KexAlgorithms curve25519-sha256
Ciphers aes256-ctr
MACs hmac-sha2-256
HostKeyAlgorithms ssh-ed25519
Subsystem sftp internal-sftp
```

No se consideró suficiente comprobar únicamente que estos valores se encontraran escritos dentro del archivo. Para verificar que OpenSSH había cargado realmente la configuración se ejecutó:

```
docker exec lab04-ssh-server sshd -T
```

Entre los parámetros relevantes se obtuvo:

```
ciphers aes256-ctr
macs hmac-sha2-256
kexalgorithms curve25519-sha256
hostkeyalgorithms ssh-ed25519
hostkey /etc/ssh/ssh_host_ed25519_key
```

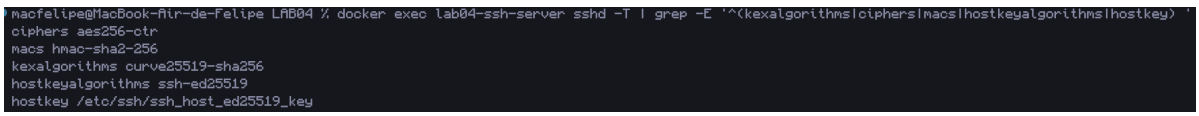


Figura 2: Verificación de la configuración criptográfica efectiva cargada por OpenSSH.

El resultado permitió comprobar que los cuatro mecanismos solicitados se encontraban efectivamente cargados por el servidor.

3.1.3. Explicación de los algoritmos utilizados

Los algoritmos configurados cumplen funciones diferentes dentro del establecimiento y protección de una sesión SSH.

- `curve25519-sha256`: corresponde al mecanismo utilizado durante el intercambio de claves. Permite que cliente y servidor establezcan material secreto compartido sin transmitir directamente la clave de sesión a través de la red.
- `aes256-ctr`: proporciona confidencialidad al contenido transmitido después del establecimiento de las claves. Utiliza AES con una clave de 256 bits en modo contador (CTR).
- `hmac-sha2-256`: permite verificar la integridad y autenticidad de los mensajes transmitidos. Su función es detectar modificaciones realizadas sobre los datos durante el transporte.
- `ssh-ed25519`: corresponde al algoritmo utilizado por la clave de host del servidor. Permite al cliente verificar criptográficamente la identidad del servidor antes de continuar con la sesión.

Por lo tanto, los cuatro mecanismos cumplen funciones complementarias: intercambio de claves, autenticación del servidor, confidencialidad e integridad de la comunicación.

3.1.4. Conexión cliente-servidor

Una vez iniciado el contenedor se estableció una conexión desde el computador cliente mediante:

```
ssh -p 2222 felipe@localhost
```

Dentro del servidor se utilizaron los siguientes comandos para comprobar el usuario, el nombre del contenedor y los extremos de la conexión:

```
whoami
hostname
echo $SSH_CONNECTION
```

La variable `SSH_CONNECTION` mostró:

```
192.168.65.1 44244 172.17.0.2 22
```

Posteriormente se realizó una nueva conexión utilizando el modo detallado del cliente OpenSSH:

```
ssh -vv -p 2222 felipe@localhost
```

La salida confirmó los algoritmos seleccionados para la sesión:

```
kex: algorithm: curve25519-sha256
kex: host key algorithm: ssh-ed25519
kex: server->client cipher: aes256-ctr MAC: hmac-sha2-256
kex: client->server cipher: aes256-ctr MAC: hmac-sha2-256
```

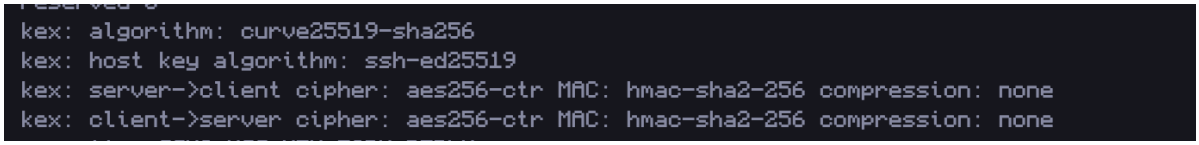


Figura 3: Algoritmos negociados durante la conexión SSH observados desde el cliente.

La coincidencia entre estos resultados y la configuración efectiva del servidor permitió comprobar que los algoritmos definidos fueron utilizados realmente durante la conexión.

3.1.5. Análisis de tráfico

Para analizar el establecimiento de la sesión SSH se inició una captura en Wireshark antes de realizar una nueva conexión.

Debido a que el servidor estaba publicado mediante `localhost`, la captura se realizó sobre la interfaz de loopback del computador.

El tráfico relacionado con la conexión se aisló mediante el siguiente filtro:

```
tcp.port == 2222
```

Identificación de las versiones SSH Durante la etapa inicial de la conexión fue posible observar la cadena de identificación enviada por el cliente:

```
SSH-2.0-OpenSSH_10.5
```

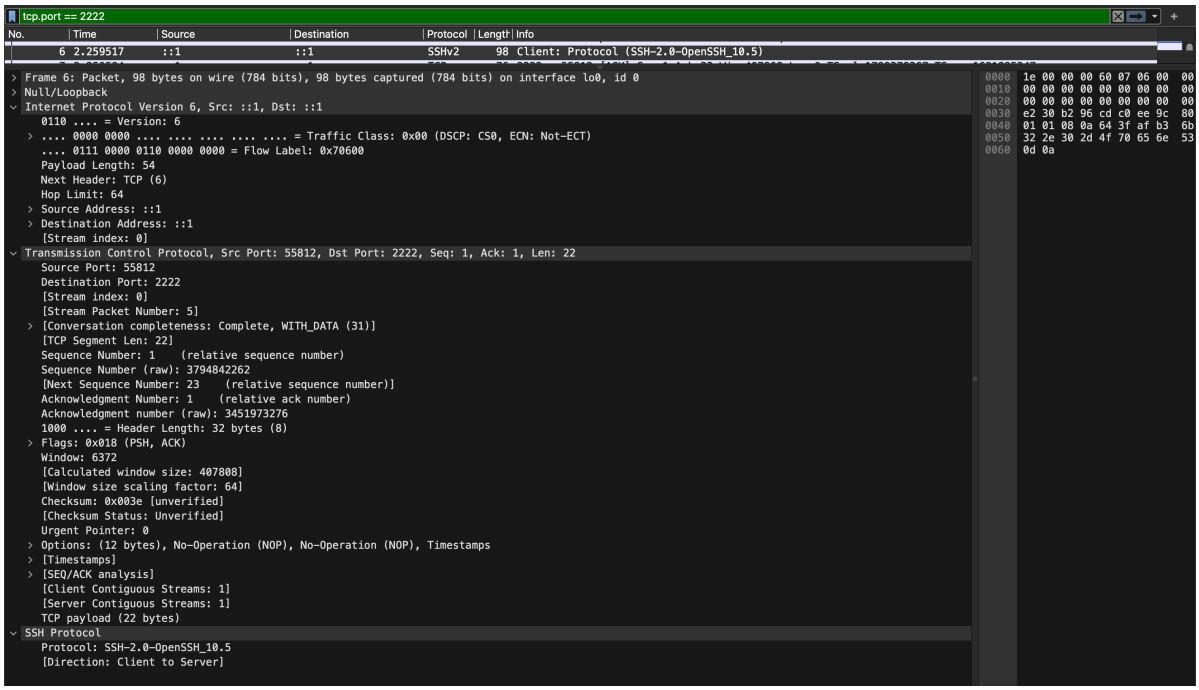


Figura 4: Identificación de la versión del cliente SSH observada en Wireshark.

También se observó la identificación enviada por el servidor:

```
SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.20
```

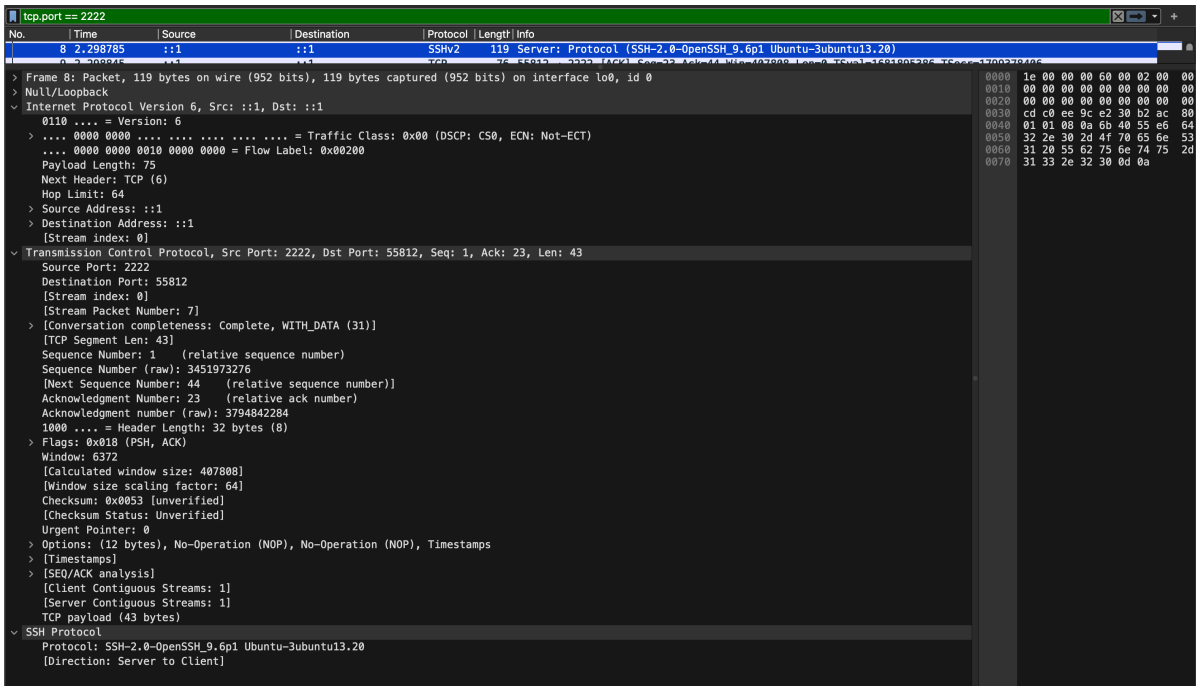


Figura 5: Identificación de la versión del servidor SSH observada en Wireshark.

Esto demuestra que las versiones del protocolo y de la implementación OpenSSH utilizada pueden ser identificadas por un tercero con acceso al tráfico.

Algoritmos criptográficos Posteriormente se analizó el mensaje `SSH_MSG_KEXINIT`, correspondiente a la negociación inicial.

Dentro de este paquete se identificaron los mecanismos configurados:

```
Key Exchange: curve25519-sha256
Host Key: ssh-ed25519
Encryption: aes256-ctr
MAC: hmac-sha2-256
```

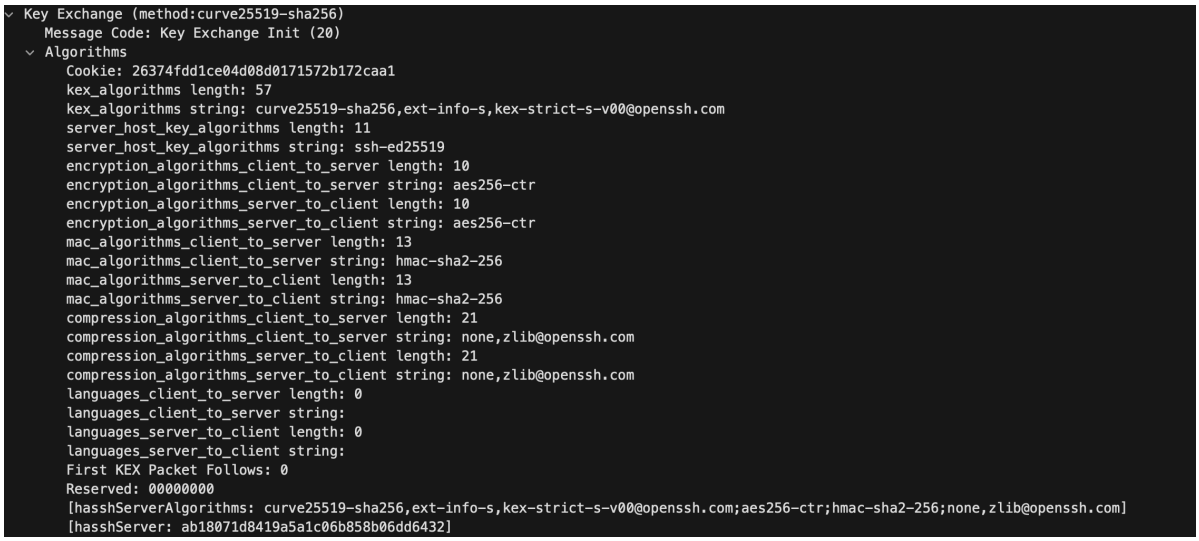


Figura 6: Algoritmos criptográficos observados durante el intercambio inicial de SSH.

Por lo tanto, un tercero puede conocer los mecanismos criptográficos utilizados durante la negociación aun cuando no disponga de las claves de sesión.

Contenido cifrado Después de completar el intercambio de claves se observó el mensaje `SSH_MSG_NEWKEYS`.

A partir de este momento Wireshark comenzó a identificar los paquetes posteriores como contenido cifrado:

```
New Keys
Encrypted Packet
MAC
```

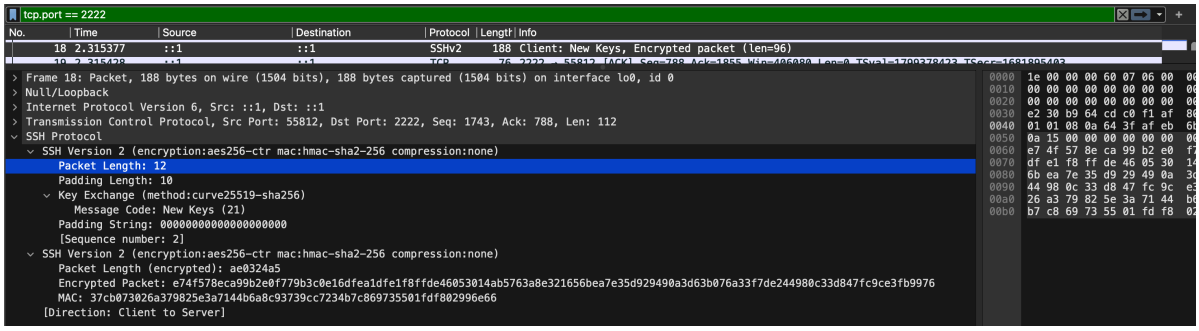


Figura 7: Tráfico cifrado observado después del establecimiento de las claves SSH.

Durante la sesión se ejecutaron comandos de prueba como:

```
whoami
hostname
echo "LAB04 prueba SSH"
```

Sin embargo, estas cadenas no aparecieron directamente en texto legible dentro de los paquetes posteriores al establecimiento de las claves.

A partir del análisis se identificaron al menos tres elementos visibles para un tercero:

1. **Direcciones, puertos y características del tráfico.**

   Es posible conocer los extremos de la comunicación, los puertos utilizados, el tamaño de los paquetes, la dirección del tráfico y los momentos en que son transmitidos.

2. **Versiones del protocolo y del software SSH.**

   Las cadenas de identificación intercambiadas al inicio permiten conocer las versiones de OpenSSH utilizadas por cliente y servidor.

3. **Algoritmos criptográficos negociados.**

   Los mensajes iniciales permiten identificar los mecanismos utilizados para intercambio de claves, autenticación del servidor, cifrado e integridad.

En cambio, una vez activadas las claves de sesión no fue posible visualizar directamente la contraseña, los comandos ejecutados, sus resultados ni el contenido transmitido.

Por lo tanto, SSH protege la confidencialidad del contenido de la sesión, pero determinados metadatos asociados a la comunicación permanecen observables.

3.2. Actividad 2

3.2.1. Identificación del paquete RSA_PIN

Para la segunda actividad se analizó el archivo `transacciones_cifradas.pcap` proporcionado para el laboratorio.

En primer lugar se identificó el paquete que contenía el PIN cifrado mediante el siguiente filtro:

```
tcp contains ‘‘RSA_PIN’’
```

El filtro permitió identificar un único paquete cuyo contenido comenzaba con:

```
RSA_PIN|
```

La comunicación observada correspondía a:

```
Origen: 10.10.10.10:49152
Destino: 10.10.10.20:4444
```

El payload TCP tenía una longitud total de 280 bytes.

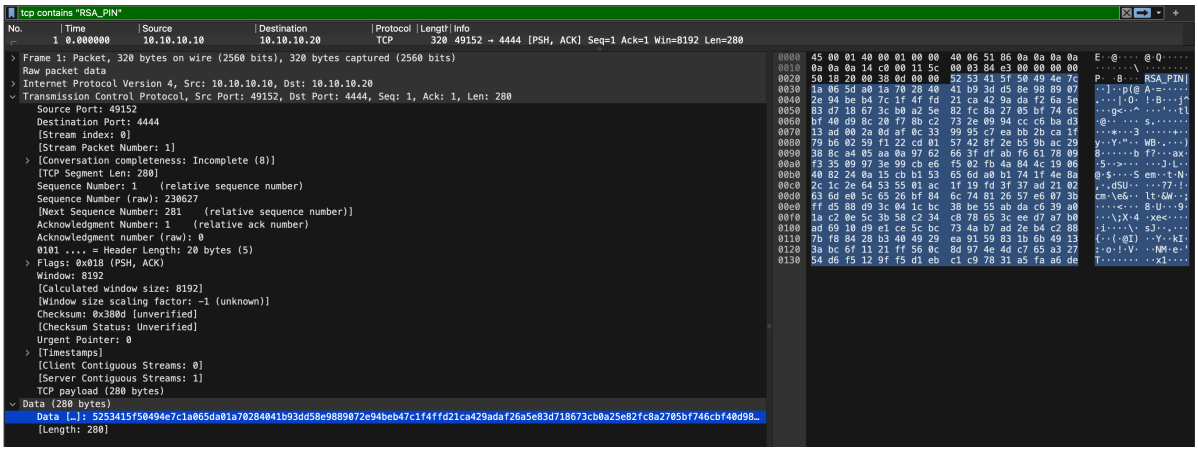


Figura 8: Identificación del paquete RSA_PIN dentro de la captura proporcionada.

La estructura identificada fue:

Campo	Longitud	Posición
RSA_PIN|	8 bytes	0 a 7
Salt	16 bytes	8 a 23
PIN cifrado mediante RSA	256 bytes	24 a 279

Los 256 bytes correspondientes al ciphertext son coherentes con el tamaño de una operación realizada utilizando una clave RSA de 2048 bits.

Posteriormente se utilizó el script desarrollado en Python para extraer estos campos automáticamente.

El programa entregó:

```
Paquetes totales en la captura: 41
Paquetes RSA_PIN encontrados: 1
Paquetes AESGCM encontrados: 40
Longitud payload RSA_PIN: 280 bytes

=== RSA_PIN ===
Etiqueta: RSA_PIN|
Salt: 1a065da01a70284041b93dd58e988907
Longitud salt: 16 bytes
Longitud PIN cifrado: 256 bytes
```

```
macfelipe@MacBook-Air-de-Felipe LAB04 % python3 Actividad2/descifrar_transacciones.py
Paquetes totales en la captura: 41
Paquetes RSA_PIN encontrados: 1
Paquetes AESGCM encontrados: 40
Longitud payload RSA_PIN: 280 bytes

=== RSA_PIN ===
Etiqueta: RSA_PIN|
Salt: 1a065da01a70284041b93dd58e988907
Longitud salt: 16 bytes
Longitud PIN cifrado: 256 bytes
```

Figura 9: Extracción mediante Python de los campos presentes en el paquete RSA_PIN.

3.2.2. Identificación de los paquetes AESGCM

A continuación se identificaron los paquetes correspondientes a las transacciones protegidas mediante AES-GCM.

Para ello se utilizó:

```
tcp contains ‘‘AESGCM’’
```

El filtro permitió identificar 40 paquetes cuyo contenido comenzaba con:

```
AESGCM|
```

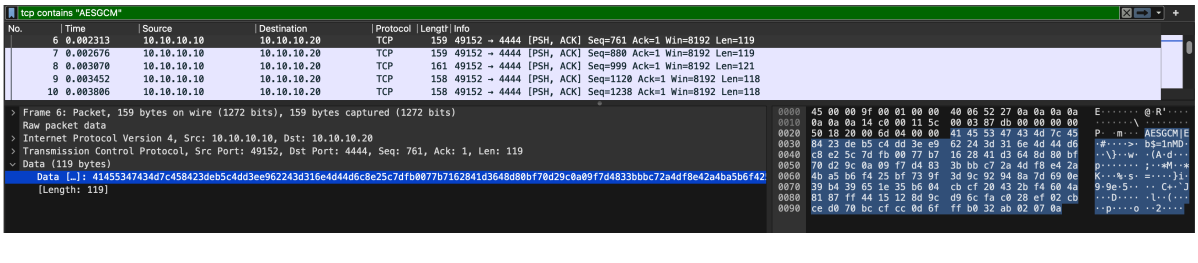


Figura 10: Identificación de los paquetes correspondientes a transacciones AES-GCM.

La estructura utilizada por cada paquete fue:

Campo	Longitud
AESGCM|	7 bytes
Nonce	12 bytes
Ciphertext + tag GCM	Longitud variable

En el primer paquete analizado se obtuvo el siguiente nonce:

```
2f0f7c0cec6fc6216066c11e
```

Su longitud fue de 12 bytes, coincidiendo con la estructura indicada para la actividad.

3.2.3. Recuperación del PIN y derivación de la clave AES

Una vez extraído el ciphertext RSA se utilizó la clave privada proporcionada en el archivo `clave_privada.pem`.

Al cargar la clave se comprobó que correspondía a una clave RSA de 2048 bits.

El PIN fue recuperado mediante RSA-OAEP utilizando SHA-256 como función hash principal y dentro de MGF1.

La operación implementada en Python fue:

```
pin_bytes = clave_privada.decrypt(
   pin_cifrado,
   padding.OAEP(
      mgf=padding.MGF1(algorithm=hashes.SHA256()),
      algorithm=hashes.SHA256(),
      label=None
   )
)
```

El resultado obtenido fue:

```
=== DESCIFRADO RSA ===
Tamaño clave RSA: 2048 bits
Algoritmo: RSA-OAEP con SHA-256
PIN recuperado: 4827
Longitud PIN: 4 dígitos
```

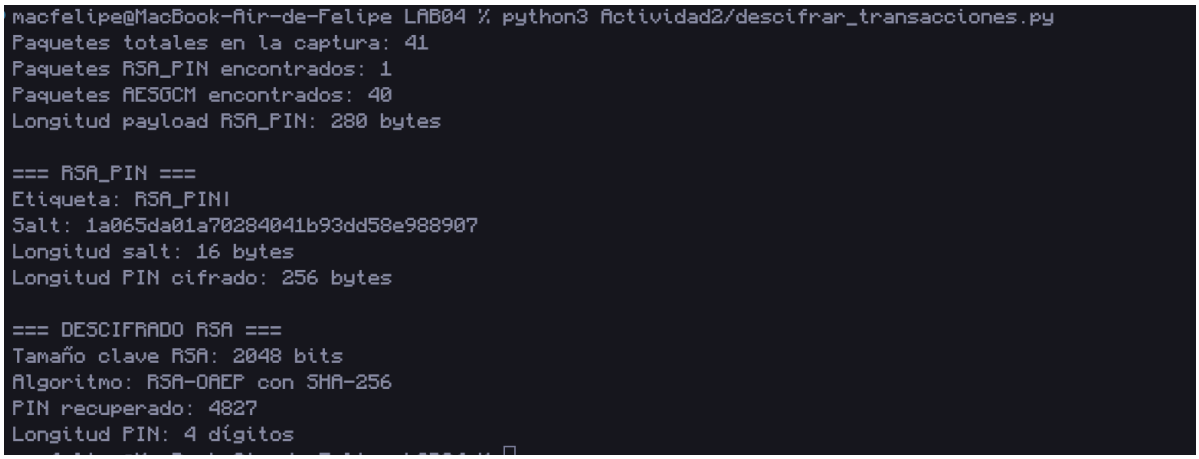


Figura 11: Recuperación del PIN utilizando RSA-OAEP con SHA-256.

Por lo tanto, el PIN recuperado fue:

4827

Posteriormente se utilizó este valor junto con la salt extraída anteriormente para derivar una clave AES de 256 bits mediante Scrypt.

Los parámetros utilizados fueron:

```
N = 16384
r = 8
p = 1
Longitud = 32 bytes
```

La derivación fue implementada mediante:

```
kdf = Scrypt(
   salt=salt,
   length=32,
   n=2**14,
   r=8,
   p=1
)

clave_aes = kdf.derive(pin_bytes)
```

El programa obtuvo:

```
=== DERIVACIÓN DE CLAVE AES ===
Algoritmo: Scrypt
N: 16384
r: 8
p: 1
Salt: 1a065da01a70284041b93dd58e988907
Longitud clave AES: 32 bytes
Clave AES-256:
f64d739ad8bd559fcd4bde7b9aecb83f
f28d200331da635d7409b7133b16eab7
```

```
macfelipe@MacBook-Air-de-Felipe LAB04 % python3 Actividad2/descifrar_transacciones.py
Paquetes totales en la captura: 41
Paquetes RSA_PIN encontrados: 1
Paquetes AESGCM encontrados: 40
Longitud payload RSA_PIN: 280 bytes

=== RSA_PIN ===
Etiqueta: RSA_PIN|
Salt: 1a065da01a70284041b93dd58e988907
Longitud salt: 16 bytes
Longitud PIN cifrado: 256 bytes

=== DESCIFRADO RSA ===
Tamaño clave RSA: 2048 bits
Algoritmo: RSA-OAEP con SHA-256
PIN recuperado: 4827
Longitud PIN: 4 dígitos

=== DERIVACIÓN DE CLAVE AES ===
Algoritmo: Scrypt
N: 16384
r: 8
p: 1
Salt: 1a065da01a70284041b93dd58e988907
Longitud clave AES: 32 bytes
Clave AES-256: f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7
```

Figura 12: Derivación de la clave AES de 256 bits mediante Scrypt.

La función de la salt es introducir un valor adicional al proceso de derivación de la clave. Esto evita que el resultado dependa únicamente del PIN. Por ejemplo, utilizar un mismo PIN con una salt diferente produce una clave derivada distinta.

Además, para reconstruir la clave AES original es necesario utilizar exactamente el mismo PIN, la misma salt y los mismos parámetros `N`, `r` y `p`.

Si cualquiera de estos valores cambia, la clave derivada también cambia. En ese caso AES-GCM no puede verificar correctamente la autenticidad ni recuperar el contenido original de las transacciones.

3.2.4. Descifrar los mensajes

Una vez obtenida la clave AES-256 se procesaron automáticamente los 40 paquetes identificados previamente.

Para cada paquete se separaron la etiqueta, el nonce de 12 bytes y el ciphertext junto con el tag:

```
etiqueta_aes = payload_aes[:7]
nonce = payload_aes[7:19]
ciphertext_y_tag = payload_aes[19:]
```

Posteriormente se realizó el descifrado mediante AES-GCM:

```
aesgcm = AESGCM(clave_aes)

mensaje_descifrado = aesgcm.decrypt(
   nonce,
   ciphertext_y_tag,
   None
)
```

La primera transacción recuperada fue:

```
=== PRIMERA TRANSACCIÓN AES-GCM ===
Etiqueta: AESGCM|
Longitud nonce: 12 bytes
Nonce: 2f0f7c0cec6fc6216066c11e
Longitud ciphertext + tag: 100 bytes

Autenticidad AES-GCM: VERIFICADA

TRANSACCION COMPLETADA
ID=TRX-426477
DE=Diego Ramirez
A=Constanza Vidal
MONTO=763923
```

```
macfelipe@MacBook-Air-de-Felipe LAB04 % python3 Actividad2/descifrar_transacciones.py
Paquetes totales en la captura: 41
Paquetes RSA_PIN encontrados: 1
Paquetes AESGCM encontrados: 40
Longitud payload RSA_PIN: 280 bytes

=== RSA_PIN ===
Etiqueta: RSA_PIN|
Salt: 1a065da01a70284041b93dd58e988907
Longitud salt: 16 bytes
Longitud PIN cifrado: 256 bytes

=== DESCIFRADO RSA ===
Tamaño clave RSA: 2048 bits
Algoritmo: RSA-OAEP con SHA-256
PIN recuperado: 4827
Longitud PIN: 4 dígitos

=== DERIVACIÓN DE CLAVE AES ===
Algoritmo: Scrypt
N: 16384
r: 8
p: 1
Salt: 1a065da01a70284041b93dd58e988907
Longitud clave AES: 32 bytes
Clave AES-256: f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7

=== PRIMERA TRANSACCIÓN AES-GCM ===
Etiqueta: AESGCM|
Longitud nonce: 12 bytes
Nonce: 2f0f7c0cec6fc6216066c11e
Longitud ciphertext + tag: 100 bytes

Autenticidad AES-GCM: VERIFICADA
Mensaje recuperado: TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923
```

Figura 13: Primera transacción recuperada y autenticada mediante AES-GCM.

Una vez comprobado el procedimiento con el primer paquete, el proceso se automatizó para las 40 transacciones.

El resultado final obtenido fue:

```
======================================
RESUMEN
======================================
Paquetes AESGCM encontrados: 40
Transacciones recuperadas: 40
Transacciones verificadas correctamente: 40

Resultados guardados en: transacciones_recuperadas.csv
```

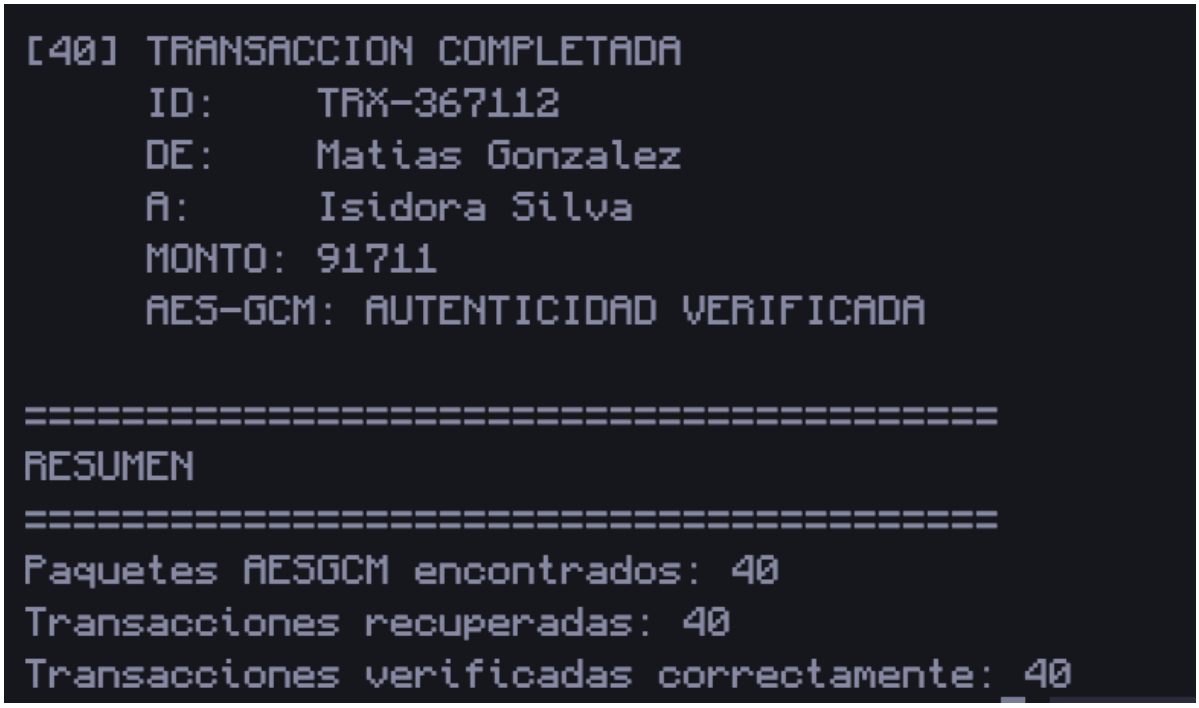


Figura 14: Resultado final del descifrado y verificación de las 40 transacciones.

La recuperación y verificación correcta de los 40 mensajes permitió comprobar que la clave derivada mediante Scrypt correspondía a la utilizada durante el cifrado original.

AES-GCM permitió además verificar la autenticidad de cada mensaje mediante el tag asociado a cada ciphertext.

Los resultados fueron guardados también en el archivo `transacciones_recuperadas.csv`.

Transacciones recuperadas A continuación se presentan las transacciones recuperadas desde la captura.

ID	Remitente	Destinatario	Monto
TRX-426477	Diego Ramirez	Constanza Vidal	763923
TRX-346837	Catalina Vargas	Valentina Rojas	58911
TRX-412154	Sebastian Munoz	Constanza Vidal	485213
TRX-071484	Felipe Morales	Tomas Sepulveda	314718
TRX-805221	Isidora Silva	Javiera Fuentes	104470
TRX-971697	Valentina Rojas	Camila Torres	785714
TRX-139273	Catalina Vargas	Nicolas Herrera	913757
TRX-652114	Camila Torres	Fernanda Araya	855548
TRX-432361	Vicente Soto	Matias Gonzalez	818686
TRX-818929	Antonia Castro	Tomas Sepulveda	885868

Tabla 1: Transacciones recuperadas, registros 1 al 10.

ID	Remitente	Destinatario	Monto
TRX-871181	Catalina Vargas	Camila Torres	514444
TRX-157566	Tomas Sepulveda	Javiera Fuentes	436301
TRX-186243	Vicente Soto	Daniela Reyes	698110
TRX-121024	Isidora Silva	Sebastian Munoz	623481
TRX-339097	Matias Gonzalez	Constanza Vidal	77226
TRX-663549	Isidora Silva	Antonia Castro	576901
TRX-367001	Fernanda Araya	Tomas Sepulveda	282567
TRX-188807	Camila Torres	Antonia Castro	195173
TRX-096936	Felipe Morales	Antonia Castro	359917
TRX-695623	Catalina Vargas	Nicolas Herrera	619299

Tabla 2: Transacciones recuperadas, registros 11 al 20.

ID	Remitente	Destinatario	Monto
TRX-407749	Camila Torres	Sebastian Munoz	182369
TRX-069658	Valentina Rojas	Javiera Fuentes	556520
TRX-232469	Felipe Morales	Tomas Sepulveda	23769
TRX-940466	Vicente Soto	Fernanda Araya	292293
TRX-755002	Matias Gonzalez	Daniela Reyes	946989
TRX-962321	Felipe Morales	Daniela Reyes	589780
TRX-324981	Diego Ramirez	Daniela Reyes	971045
TRX-337011	Antonia Castro	Felipe Morales	777198
TRX-995006	Vicente Soto	Sebastian Munoz	587590
TRX-630572	Camila Torres	Sebastian Munoz	332702

Tabla 3: Transacciones recuperadas, registros 21 al 30.

ID	Remitente	Destinatario	Monto
TRX-517969	Catalina Vargas	Daniela Reyes	484770
TRX-447236	Matias Gonzalez	Constanza Vidal	331352
TRX-852645	Constanza Vidal	Felipe Morales	343065
TRX-188241	Camila Torres	Constanza Vidal	781107
TRX-713923	Diego Ramirez	Catalina Vargas	21358
TRX-362125	Camila Torres	Sebastian Munoz	154118
TRX-961549	Felipe Morales	Sebastian Munoz	614169
TRX-965060	Tomas Sepulveda	Antonia Castro	778912
TRX-437498	Valentina Rojas	Matias Gonzalez	440902
TRX-367112	Matias Gonzalez	Isidora Silva	91711

Tabla 4: Transacciones recuperadas, registros 31 al 40.

3.2.5. Análisis respecto de criptografía post-cuántica

La pregunta final plantea si sería posible utilizar las claves pública y privada RSA proporcionadas para descifrar mensajes protegidos mediante criptografía post-cuántica.

La respuesta es **no**.

Las claves utilizadas en el laboratorio fueron generadas específicamente para RSA y poseen una estructura matemática asociada a este algoritmo. En este caso fue posible recuperar el PIN debido a que había sido cifrado utilizando RSA-OAEP y se disponía de la clave privada RSA correspondiente.

Los algoritmos de criptografía post-cuántica utilizan mecanismos matemáticos diferentes y requieren sus propias estructuras de claves. Por ejemplo, ML-KEM es un mecanismo post-cuántico de encapsulación de claves basado en problemas relacionados con retículas.

Por esta razón, una clave privada RSA no puede utilizarse como clave privada de ML-KEM ni permite recuperar directamente un secreto generado mediante dicho mecanismo.

Para recuperar información protegida mediante un esquema post-cuántico sería necesario disponer de la clave privada correspondiente específicamente al algoritmo utilizado.

Además, RSA corresponde a uno de los esquemas de clave pública que podrían verse comprometidos frente a un computador cuántico suficientemente potente. Esta es precisamente una de las razones que ha impulsado el desarrollo de algoritmos criptográficos resistentes a ataques cuánticos.

Por lo tanto, aunque las claves RSA proporcionadas permitieron recuperar el PIN cifrado mediante RSA-OAEP en este laboratorio, no serían compatibles directamente con un esquema PQC diferente.

Conclusiones y comentarios

El desarrollo del laboratorio permitió analizar de forma práctica dos escenarios relacionados con la protección criptográfica de información.

En la primera actividad se implementó un servidor SSH mediante Docker y se configuraron explícitamente los algoritmos utilizados durante la comunicación. Las verificaciones realizadas desde el servidor, el cliente y Wireshark permitieron comprobar el uso de `curve25519-sha256`, `ssh-ed25519`, `aes256-ctr` y `hmac-sha2-256`.

El análisis de tráfico mostró que SSH no oculta completamente la existencia ni las características generales de una comunicación. Un tercero todavía puede identificar elementos como los extremos de la conexión, los puertos utilizados, las versiones de OpenSSH y los algoritmos negociados. Sin embargo, una vez establecidas las claves de sesión, los comandos y el contenido transmitido dejaron de ser visibles directamente.

En la segunda actividad se analizó una captura que combinaba criptografía asimétrica y simétrica. Mediante RSA-OAEP con SHA-256 se recuperó el PIN `4827`. Posteriormente, este valor se utilizó junto con la salt extraída desde la captura y los parámetros definidos para Scrypt, obteniendo una clave AES de 256 bits.

La clave derivada permitió recuperar correctamente las 40 transacciones protegidas mediante AES-GCM. Además, todas las transacciones superaron la verificación de autenticidad,

confirmando tanto la correcta derivación de la clave como el procedimiento de descifrado implementado.

El laboratorio permitió relacionar conceptos que normalmente se estudian de forma separada, como intercambio de claves, autenticación, criptografía asimétrica, derivación de claves y cifrado simétrico autenticado. También permitió comprobar que una clave criptográfica se encuentra asociada al algoritmo para el cual fue generada y no puede reutilizarse directamente en un esquema post-cuántico diferente.

4. Bibliografia

- Captura y Claves.
- Repositorio.
- Ylonen, T. y Lonvick, C., *The Secure Shell (SSH) Transport Layer Protocol*, RFC 4253. Disponible en: `https://www.rfc-editor.org/rfc/rfc4253`.
- Adamantiadis, A. et al., *Secure Shell (SSH) Key Exchange Method Using Curve25519 and Curve448*, RFC 8731. Disponible en: `https://www.rfc-editor.org/rfc/rfc8731`.
- Bellare, M. et al., *The Secure Shell (SSH) Transport Layer Encryption Modes*, RFC 4344. Disponible en: `https://www.rfc-editor.org/rfc/rfc4344`.
- Bider, D., *SHA-2 Data Integrity Verification for the Secure Shell Transport Layer Protocol*, RFC 6668. Disponible en: `https://www.rfc-editor.org/rfc/rfc6668`.
- Josefsson, S. y Liusvaara, I., *Ed25519 and Ed448 Public Key Algorithms for the Secure Shell Protocol*, RFC 8709. Disponible en: `https://www.rfc-editor.org/rfc/rfc8709`.
- Moriarty, K. et al., *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017. Disponible en: `https://www.rfc-editor.org/rfc/rfc8017`.
- Percival, C. y Josefsson, S., *The scrypt Password-Based Key Derivation Function*, RFC 7914. Disponible en: `https://www.rfc-editor.org/rfc/rfc7914`.
- National Institute of Standards and Technology, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D. Disponible en: `https://csrc.nist.gov/pubs/sp/800/38/d/final`.
- National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203. Disponible en: `https://csrc.nist.gov/pubs/fips/203/final`.